# CareerConnect: AI Usage Log, Sprint 3

Team Member: Vlad Shkuratov | Tool: Claude Code | Session: November 8, 2026

Prompts below were cleaned up for readability (spelling and casual wording). Their meaning was not changed and no prompts were added. Pasted GitHub issue text is summarized instead of copied in full. Routine Git questions are not logged because the AI output did not change any deliverable.

## Sprint 3 Session (November 8, 2026)

### Task 3.1: AI Interview Preparation Backend and API: US-14.01 (Issue #132)

**Purpose of AI Use**
Implementation and debugging help for my assigned issue, the backend for our team's AI feature.

**Prompt / Response**

> ***Prompt:*** *Implement issue #132 using the acceptance criteria and tasks below. (Issue text pasted.)*
>
> ***Follow-up 1:*** *I added a Groq API key to my .env. Test the feature with real AI calls, and only implement what the issue asks for. Remove anything extra.*
>
> ***Follow-up 2:*** *(Project board screenshot attached.) If something belongs to #132 and is not its own issue on the board, keep it. Make sure everything works as required.*

**Response:** Claude followed the existing project structure and produced a first version. Based on my follow-ups, it removed work that belonged to other issues and kept the message limits, which protect the new endpoint and are not tracked anywhere else.

**Follow-up Questions (code review session after implementation)**

> ***Q:*** *Why did we choose Groq, and what happens if a model gets retired like llama-3.1-8b-instant did?*

**A:** Groq has a free tier with no credit card and an OpenAI-compatible API, so it fits a student project. The model name is read from AI_MODEL, so a retired model can be replaced in .env without a code change. A retired model makes the endpoints return a 502 with the provider's error instead of crashing.

> ***Q:*** *How does the backend stop a user from overriding the system prompt or changing the AI's rules?*

**A:** The system prompt is always built on the server. The client can only send messages with the role user or assistant, so a message with the role system is rejected with a 422. The rules in the prompt also tell the model to decline requests to ignore its instructions, but that part is prompt-based and not guaranteed.

> ***Q:*** *Why does the app return a 503 when AI_API_KEY is missing instead of crashing at startup, like it does for JWT_SECRET_KEY?*

**A:** Login cannot work at all without JWT_SECRET_KEY, so failing early is correct there. The AI feature is optional, so teammates without a Groq key can still run the rest of the app, and only the interview endpoints return a 503 with a clear message.

> ***Q:*** *How does the resume PDF get into the prompt, and what happens if the PDF is corrupt or missing?*

**A:** The backend picks the user's default resume (or the newest one), extracts its text with pypdf and puts it in the system prompt. If the file is missing, corrupt or empty, the text is empty and the prompt says there is no resume, so the AI works from the job posting alone.

> ***Q:*** *Why are the resume and job description cut to a maximum length, and why are only the last 20 messages sent?*

**A:** Every request resends the whole prompt and history, so the limits keep each request small. That keeps responses fast and stays within the free tier's token limits.

**AI-Suggested Content**
Use Groq's free API. Extract the resume text with pypdf and build the system prompt from the job posting and the resume, with rules to decline off-topic questions. Two endpoints (opening message and chat), a 503 when no key is set so teammates without a key can still run the app, and limits of 20 messages and 2000 characters per message.

**Validation**
I set up the API key and had the feature tested live: on-topic and off-topic questions, "ignore your instructions", a fake system message (rejected with 422), off-topic questions later in a conversation, no resume, a corrupt PDF and long answers. The existing 52 tests passed after each change. I reviewed the results before committing.

**Decision**
Modified before use. I had the extra work removed: a test file belonging to #133 and a configurable provider URL. The suggested model (llama-3.1-8b-instant) failed with my key, so we switched to openai/gpt-oss-20b. Live testing showed the AI claiming skills that were not on the resume and cutting off long answers, so a prompt rule was added and the token limit raised.

**Reflection**
Testing with a real key caught problems that mocked tests would miss. Keeping the AI inside the issue's scope was important, because its first version included work assigned to teammates.

**Responsible Person**
Vlad Shkuratov

## Task 3.2: AI Interview Preparation UI: US-14.03 (Issue #134)

**Purpose of AI Use**
Implementation help for the UI. I took this issue right after #132 so the backend and UI would be finished and tested together.

**Prompt / Response**

> ***Prompt:*** *Next, implement #134, the interview prep UI (issue text pasted), so both parts of the feature are done together.*

**Response:** Claude pointed out that #134 requires saved past sessions, while the #132 endpoints did not store anything. Instead of adding a second set of endpoints, the #132 endpoints were changed to save sessions.

**Follow-up Questions (code review session after implementation)**

> ***Q:*** *Why did the #132 endpoints have to change to support past sessions?*

**A:** They did not store anything, and #134 requires opening past sessions. So sessions and their messages are now saved in the database, under /api/interview/sessions.

> ***Q:*** *Why does the AI call happen before anything is saved to the database?*

**A:** If the AI call fails, nothing is saved, so the conversation never ends up with a user message that has no reply.

> ***Q:*** *Why does resume_id on InterviewSession have no foreign key, when job_id does?*

**A:** Users can delete resumes. Without a foreign key, deleting a resume does not break or block the user's old interview sessions.

*Q: What does the conversation page do if sending a message fails?*

**A:** It shows the error and puts the message back in the input box so the user can retry without retyping it.

*Q: How does the backend stop one user from opening another user's session?*

**A:** Every session lookup is filtered by the logged-in user's id. Another user's session is treated as not found and returns a 404.

**AI-Suggested Content**
Two tables (InterviewSession, InterviewSessionMessage) and endpoints under /api/interview/sessions. Three pages with no new libraries: /interview (past sessions), /interview/new (pick an application) and /interview/{id} (conversation). An Interview Prep link in the navbar. If the AI call fails, nothing is saved and the user's message is put back in the input box.

**Validation**
Tested in the browser with real AI replies: new session, messages, refresh, reopening a session, an off-topic question and a phone-sized screen. No console or server errors, and the build, lint and existing tests passed. Screenshots showed raw ** symbols and sideways scrolling on phones, and both were fixed.

**Decision**
Accepted. I listed one known limitation in the PR: the AI sometimes invents results in sample answers (one test produced "a 30% drop in help-desk tickets").

**Reflection**
The UI requirement changed the backend design, so planning both issues together from the start would have avoided a rewrite. Invented numbers are a real risk for an interview coach, so I want to raise it with the team before the demo.

**Responsible Person**
Vlad Shkuratov

## Task 3.3: Interview Preparation Unit Tests: US-14.02 (Issue #133)

**Purpose of AI Use**
Test case generation. After opening the PR for #132 and #134, I noticed #133 was still open.

**Prompt / Response**

***Prompt:*** *We still need #133 (issue text pasted). Write the unit tests so I can add them as a new commit on the same branch.*

**Response:** Following the issue's note that real AI answers cannot be tested, the tests cover prompt generation and context passing only.

**Follow-up Questions (code review session after implementation)**

*Q: How do the tests check the prompt without calling the real AI or needing a key in CI?*

**A:** The network call is replaced with a fake that returns a fixed reply, so no request leaves the test and no key is needed.

*Q: What does ai_requests record, and why replace httpx.post rather than the whole service function?*

**A:** It records the payload of every request that would have been sent to the AI. Replacing only httpx.post means the real prompt-building code still runs, so the tests check the actual prompt and history the model would receive.

*Q: Why does one test send three messages and check the off-topic rule on every request?*

**A:** The issue requires the rule to apply to further discussion, not just the first message. Checking every request proves the rule is resent each turn.

*Q: How do we know these tests would actually fail if the code broke?*

**A:** The code was broken on purpose four ways (off-topic rule removed, job title removed, history dropped, wrong job used), and each time at least one test failed.

**AI-Suggested Content**
tests/test_interview.py with 9 tests: the prompt contains the right posting and not another job's, the resume text is included (or the prompt says there is none), a new session uses the selected application's job, and the off-topic rule and conversation history are sent on every later message, in order. The AI call is faked, so CI needs no key.

**Validation**
All 9 tests passed and the full suite reached 61. To prove the tests can fail, the code was broken on purpose four ways (no off-topic rule, no job title, no history, wrong job), and each break made tests fail. The code was restored, and only the test file changed.

**Decision**
Accepted.

**Reflection**
Breaking the code on purpose is a quick way to check that AI-written tests actually test something.

**Responsible Person**
Vlad Shkuratov

## Task 3.4: Analytics Page UI: US-07.04 (Issue #116)

**Purpose of AI Use**
Implementation help for the Application Analytics Dashboard, a required Sprint 3 feature.

**Prompt / Response**

***Prompt:*** *I switched back to main and created a new branch. Implement #116 (issue text pasted).*

**Response:** Claude confirmed the new branch did not include the unmerged interview code, then checked what the existing analytics endpoints return before building the page.

**Follow-up Questions (code review session after implementation)**

*Q: Why does each card manage its own dates and granularity instead of one filter for the whole page?*

**A:** The issue asks for each graph and stat to have its own period and granularity. The endpoints also differ: salary and the category and location stats only accept a date range.

*Q: How does AnalyticsCard avoid showing old results when you change the dates quickly?*

**A:** When the dates change, the card ignores the response from the earlier request, so a slow old response cannot overwrite newer results.

*Q: Why is the salary graph a range chart and not a histogram, and what would the backend need to change?*

**A:** The endpoint only returns the average, minimum, maximum and standard deviation. A histogram would need the backend to also return salaries grouped into ranges with a count for each.

*Q: Why does parsePeriod build the date by hand instead of using new Date("2026-11-08")?*

**A:** JavaScript reads that string as midnight UTC, which is the evening before in Montreal, so the chart would show the wrong day. Building it from the year, month and day keeps it in local time.

*Q: Why did we draw the charts with plain HTML and CSS instead of adding a chart library?*

**A:** Bar charts are simple enough to draw that way, and it avoids adding a new dependency to the project.

**AI-Suggested Content**

An Analytics page at /analytics with a "View analytics" button on the dashboard. Bar charts for searches, applications and viewed postings by day, week or month, with hover details and a data table. A salary chart, most frequent category, most common location and the My Applications list. Each card has its own date range. No backend changes or new libraries.

**Validation**
Tested in the browser with sample data: each card, granularity and period changes, a 366-day range, an invalid date range, empty results, refresh and a phone-sized screen. Screenshots were checked for layout problems, which found a missing year on long ranges and a syntax error. Both were fixed, and lint and the build passed.

**Decision**
Modified before use. The backend only returns the average, minimum, maximum and spread of salaries, so the salary graph is a range chart instead of a histogram. I flagged this to the team, because a real histogram would need a backend change outside this issue.

**Reflection**
Checking the backend first showed what the page could actually display. Reviewing screenshots caught problems that the automated checks did not.

**Responsible Person**
Vlad Shkuratov

## Task 3.5: Submission UI Issue Fixing: US-06.03 (Issue #131)

**Purpose of AI Use**
Planning and implementation help for the bug list in the job application flow.

**Prompt / Response**

***Prompt:*** *Here is #131 (issue text pasted). Which branch should I start from, given the interview and analytics PRs are not merged yet?*

***Follow-up 1:*** *I'm on the new branch. Go ahead with the fixes.*

**Response:** Claude recommended branching off main, since #131 does not depend on the other two branches, and warned that all three change App.jsx and might conflict when merged.

**Follow-up Questions (code review session after implementation)**

***Q:*** *What caused "Already applied" to appear right after a successful application, and how does the success check fix it?*

**A:** After submitting, the page saw that an application existed and showed the already-applied message. The success check shows the success message first when the application was just submitted in this visit.

***Q:*** *Why was a new /jobs/{id}/apply route better than the old show/hide state in JobDetailPage?*

**A:** The Apply buttons can link straight to the page, and refreshing or using the back button works, because the application page has its own URL.

***Q:*** *How does the job list know which jobs you applied to without you clicking each one?*

**A:** It loads the user's applications once and checks each job against that list to show the badge.

***Q:*** *Why does the Upload Resume button use a hidden file input and useRef?*

**A:** The browser's file input is hard to style. The button uses the ref to open the hidden input's file picker, so users see one clear button.

***Q:*** *Why is the email field still locked, and why would adding a column be risky without migrations?*

**A:** Applications have nowhere to store a different email. A new column would need a migration, and since the tables are only created when missing, existing databases would not get the column and would break.

**AI-Suggested Content**
Show only the success message after applying, and "Already Applied" when returning. Editable name field saved to the profile. One Upload Resume button that uploads and selects the file. A standalone page at /jobs/{id}/apply. Status badges on the list, preview and full posting. A greyed-out "Already applied" button. One back link instead of two.

**Validation**
Tested a full application in the browser against the real backend, including rejecting a non-PDF, uploading a PDF, submitting and returning to the job. Confirmed the application used the uploaded resume. No console or server errors and the build passed.

**Decision**
Modified before use. I kept the email field read-only: applications have nowhere to store a different email, and adding a column would break existing databases because we have no migrations. This is noted in the PR.

**Reflection**
Choosing the branch first kept unmerged work out of this PR. Some items in an issue need a team decision, not just code.

**Responsible Person**
Vlad Shkuratov

## Task 3.6: Pull Request Descriptions and Commit Messages

**Purpose of AI Use**
Documentation writing for my three PRs (#143 for interview prep, analytics, and #131).

**Prompt / Response**

> ***Prompt:*** *Write the PR description for this branch in our team's template.*
>
> ***Follow-up 1:*** *Do the same for the analytics PR, same format with the new changes.*
>
> ***Follow-up 2:*** *Same for the #131 PR.*

**Response:** Each description was written from the branch's actual diff against main.

**AI-Suggested Content**
Titles, descriptions (Summary, Changes Made, Setup, Testing, Known Limitations, Checklist) and commit messages. It also caught stray characters ("[200~") in my first commit title from pasting into the terminal.

**Validation**
Compared each description with the diff, and confirmed .env is not tracked and no API key is in the diff.

**Decision**
Accepted.

**Reflection**
Basing the description on the diff keeps it accurate. Checking for API keys is worth doing on every PR that touches one.

**Responsible Person**
Vlad Shkuratov